

1. About the Project

This project is a **Hybrid Movie Recommendation System** that offers highly personalized suggestions by combining content-based filtering, collaborative filtering, demographic filtering, and behavioral analytics. It is designed using a dataset derived from IMDb's top 1000 movies, allowing users to receive recommendations tailored to their preferences, interactions, and demographic information such as age, occupation, and mood.

The system handles both new and returning users by leveraging both cold-start strategies and historical activity logs. It ensures better recommendation precision through layered filtering: pre-filtering to reduce the search space and post-filtering to fine-tune results. The project is scalable, modular, and built for real-time recommendation use cases.

2. Tech Stack

- **Frontend:**
 - HTML (User interface)
 - CSS (Bootstrap for responsive styling)
 - JavaScript (AJAX and DOM manipulation via Vanilla JS)
 - **Backend:**
 - Python (Flask Web Framework)
 - SQLite (Lightweight embedded database for user activity and preferences)
 - **Tools and Libraries:**
 - Pandas (Data manipulation)
 - Numpy (Numerical operations)
 - Scikit-learn (Vectorization, similarity computation)
 - Flask (Web routing, request handling)
 - **Dataset:**
 - CSV file `movies_data.csv` with columns like Title, Genre, Runtime, Rating, Director, Cast, etc.
-

3. Algorithms Used

- **Content-Based Filtering:**
 - **TF-IDF Vectorization:** Used on the Overview and Genre fields to quantify textual information.
 - **Cosine Similarity:** To measure the closeness between movie vectors and the user's profile.
- **Collaborative Filtering:**
 - **User-Item Matrix (implicit):** Built from user interactions (clicks, hovers, likes).
 - **Similarity Indexing:** Manual computation of cosine similarity across user-item behavior vectors.
- **Demographic Filtering:**
 - Age, mood, and occupation-based clustering.
 - Popular genre aggregation based on demographic cohorts.

- **Popularity filter Algorithm:**
 - Ranking based on cumulative views, likes, searches.
 - Time-weighted scoring (recent interactions are prioritized).
-

4. Models Used

- **TF-IDF Vectorizer** (from `sklearn.feature_extraction.text`):
 - Used on Overview and Genre columns to convert text into numeric feature vectors.
 - **Cosine Similarity** (from `sklearn.metrics.pairwise`):
 - Applied on TF-IDF matrices for content filtering.
 - **One-Hot Encoded Demographic Model:**
 - Categorical encoding for age groups, mood, and occupation.
 - **Manual Weighted Hybrid Model:**
 - Content score + collaborative score + demographic score + popularity score → Final rank
-

5. Strategies Used

- **Multi-Faceted Hybridization:**
 - Layered recommendations with fallback logic: If collaborative filtering lacks data, fallback to content-based or demographic-based suggestions.
 - **Contextual Personalization:**
 - Mood + Age influence genre weightings.
 - Time of day (optional future feature) to personalize type (comedy vs thriller).
 - **Session-Based Tracking:**
 - Events like hover, click, view, like are logged to SQLite.
 - These build a session profile to feed into collaborative and trending models.
-

6. Pre-Filtering Techniques

- **Genre Filtering:** Direct input from user checkbox selections.
 - **Year Range:** Min and max years limit the dataset slice.
 - **Rating Filter:** Exclude movies below a threshold.
 - **Certification Filter:** PG, R, UA-based filtering for age compatibility.
 - **Demographic Initial Bias:** Set a base vector depending on age and mood before actual user interaction.
-

7. Post-Filtering Techniques

- **Re-Ranking Based on Trends:** If a movie is trending, its position is boosted.
- **Demographic Modifier:** Slight boost to genre-matching movies that align with user's profile.

- **Recency Modifier:** Movies recently released or highly interacted with are pushed up.
 - **Interaction Decay:** Past interactions fade in influence over time.
-

8. How Demographics Help

Demographic profiling helps in two major ways:

1. **Cold Start Problem:**
 - When a user has no history, demographics provide an initial weight vector for preferences.
 - For example, a 21-year-old student selecting "Energetic" mood might get action, thriller, or comedy as top genres.
 2. **Fine-Tuning Recommendations:**
 - User may have similar behavior but different demographic preferences (e.g., older users preferring classics).
 - This allows the system to tweak the final list using soft scores.
-

9. Integration: How It All Works Together

1. **Frontend (index.html & main.js):**
 - Handles user input: search bar, filters, demographics, etc.
 - Triggers AJAX calls to Flask endpoints /recommend, /search, /demo_rec, etc.
 2. **Flask Server (app.py):**
 - Handles HTTP requests, returns JSON or rendered templates.
 - Manages session history and logs behaviors.
 3. **Recommendation Engine (recommendation_engine.py):**
 - Core engine that reads movies_data.csv, builds TF-IDF and cosine similarity matrices.
 - Calculates scores from all three models (content, collaborative, demographic).
 - Applies pre-filters before computing similarities.
 - Applies post-filters before finalizing the recommendation list.
 4. **Evaluation Module (evaluate.py):**
 - Compares expected and generated recommendations.
 - Outputs precision, recall, F1 scores, and ranks to log files.
 5. **Logging System:**
 - Evaluation logs (evaluation_log_*.log) contain timestamped precision and recall evaluations.
 - Track model performance for future improvement.
-

10. Why This Project is Useful in the Real World

- **Personalized Experience:** Multiple filtering layers offer tailored suggestions.
- **Cold Start Friendly:** Demographic models assist when no user history exists.

- **Behavioral Adaptation:** Real-time tracking of interactions improves suggestions dynamically.
 - **Scalable Architecture:** Can integrate external databases, user login, and deployment to cloud.
 - **Extensible with AI:** Can integrate deep learning models for natural language queries or dynamic re-ranking.
 - **Application Domains:**
 - OTT platforms (Netflix, Prime)
 - In-flight entertainment
 - Local streaming services
-

11. Future Enhancements

- **Movie Embeddings via BERT or Doc2Vec**
 - **Streaming API for real-time interaction logging**
 - **A/B Testing Module for recommendation evaluation**
 - **Deployment on AWS/GCP for production-ready usage**
 - **Support for Multi-language movie titles and NLP translation**
-

This hybrid recommendation engine merges the best of multiple filtering techniques to provide a real-world intelligent system suitable for any movie streaming or discovery platform. Its layered approach ensures flexibility, robustness, and adaptability.